

The LinsnLED ethernet protocol

An attempt at reverse engineering

Sophie Tyalie

Version 0.1.0

Table of Contents

1. Preface	1
2. Introduction	2
2.1. Previous Research	2
3. The hardware	4
3.1. SPI	4
3.2. Ethernet	4
3.3. FPGA	5
3.4. The LED panel	5
4. The protocol	6
4.1. Ethernet [Physical]	6
4.2. Ethernet II frame	6
4.2.1. MAC addressing	7
4.3. LinsnLED Ethernet payload	7
4.4. The LinsnLED header	8
4.4.1. Fields	8
Package ID	8
Checksum [Chks]	8
Command [Cmd]	8
Command Data	9
4.5. Frame buffer data	9
4.6. MAC discovery	9
4.6.1. Data layout	10
4.6.2. Receiver packet	10
4.7. Configuring the system	10
4.7.1. Fields	11
Panel Index	11
Type	11
is_data?	11
Config Pkg ID	11
Address	12
Data	12
4.7.2. State machine	12
4.7.3. Possible Payload values	13
Temporary/Permanent Data [type=1 or type=8]	13
State Configuration [type=5]	13
Permanent Operation [type=9]	13
5. LinsnLED Memory layout	15
5.1. Fields	15
5.1.1. Offset 0x000501: Cascade Direction [CasD]	15
5.1.2. Offset 0x000508: Duty Cycle [DuCy]	15
5.1.3. Offset 0x000509: Scan Clock Lookup [ScCl]	15
5.1.4. Offset 0x000512: Drive IC [DIC]	16

5.1.5. Offset 0x000518 : ? & Image Control [ImCn]	17
5.1.6. Offset 0x000560 : Unknown 1	18
5.1.7. Offset 0x0006a0 : Data Exchange [part I]	18
5.1.8. Offset 0x000709 : Afterglow time [lower byte] [AfT-]	18
5.1.9. Offset 0x000760 : Newline Time [Newline]	18
5.1.10. Offset 0x000762 : Max afterglow time [MaxAfT]	18
5.1.11. Offset 0x000764 : Afterglow time [upper byte] [AfT+]	18
5.1.12. Offset 0x001002 : Start X	19
5.1.13. Offset 0x001004 : Start Y	19
5.1.14. Offset 0x001006 : Width	19
5.1.15. Offset 0x001008 : Height	19
5.1.16. Offset 0x0011a0 : Data Exchange [part II]	19
5.1.17. Offset 0x002000 : Gamma Curve [1st channel]	19
5.1.18. Offset 0x002208 : Global Brightness [GoBr]	19
5.1.19. Offset 0x00227f : Card Mode [CaMo]	20
5.1.20. Offset 0x002800 : Gamma Curve [2nd channel]	20
5.1.21. Offset 0x002a00 : Gamma Curve [3rd channel]	20
5.1.22. Offset 0x002c00 : Gamma Curve [4th channel]	20
5.1.23. Offset 0x003003 : RCG file path	20
5.1.24. Offset 0x003207 : Max width	21
5.1.25. Offset 0x003209 : Max height [MaxHeight]	21
5.1.26. Offset 0x00320b : Duty ratio [DuRa]	21
5.1.27. Offset 0x00320d : Row blanking time [RoBT]	21
5.1.28. Offset 0x00341f : Checksum [Chk]	21
5.2. Memory Table	22

WARNING

This work is not affiliated with LinsnLED, nor does it claim to fully describe the protocol. The purpose of the documentation here is to make the LinsnLED receivers compatible with other systems than Windows, like Linux, mikrocontrollers, ...

Chapter 2. Introduction

LED matrix displays are everywhere nowadays. From small 24/7 kiosks that show that they are open using monochrome LED panels to big advertisement screens in NY Time Square, London Piccadilly or even high-end cinemas with full color, high dynamic range images. To support such large installations, the hardware itself has become relatively cheap. With just 15€ [+ shipping from China] one can get their own 64x64 [~0.3ct per LED] / 20 x 20 cm LED panel. This remarkable price is reached by outsourcing the display logic onto an external module, making the panel itself just some power wiring and an assortment of shift-registers.

Due to their high availability, simple electrical design and low cost of entry (and tbqh 'cuz they are just very cool), there have been many attempts at controlling these using a diverse range of hardware. Some notable ones:

Adafruit's RGB Matrix Panel lib ^[1]	Controls a HUB75 style RGB matrix using and the GPIO pins of an Arduino. They also have a newer one, for the newer Arduino models. This is compatible with the LED matrix they sell
HZeller's RPi RGB LED Matrix ^[2]	Using the RPi GPIO and some clever tricks with its hardware PWM, they actually archive driving compatible panels with full rgb and 30fps.
Adafruit's FPGA LED matrix tutorial ^[3]	An honorable mention, because FPGAs are the way the professionals do it - but more on that later.

And these are just the most of recommended projects Just searching for the term "LED Matrix" on GitHub.com presents us with 6.7k results, so the general interest is quite high.

But almost all open source projects do not take full advantage of the [honestly] dumb piece of PCB they control. Due to the fact, that every driving logic resides outside the panels, using the right controller can be the difference between a monochrome RGB display with ~1 FPS refresh rate, or a 12bit HDR 120fps display using the exact same LED panel. So it's only natural, that professionals do not use their custom controller solution, but instead employ a proven off the shelf solution. And here comes our company into the larger picture: LinsnLED.

LinsnLED is one of the more well known companies which design, program and produce controller modules for the aforementioned LED panels. Their so-called receiver modules (aka receives images and controls LED panels) can be driven over a simple Ethernet cable and are fairly cheap at around 15€ per piece on AliExpress. There are many reasons why they are cool. Starting with the fact that they were one of the cheaper FPGA dev platforms^[4] out there, when FPGAs weren't really in reach yet for tinkerers around the globe. Over the fact that a single one can sometimes drive even large installations [720p], expanding on that ability by simply adding more receivers. To the reason why they have been chosen for this reverse engineering project: They are highly configurable and as such compatible with almost every LED panel out there.

Sadly there is no open source driver available so far, but only Linsn's Windows based application or their controller modules that accept HDMI, DVI or VGA input. Luckily for us there is not a lot of magic behind it, meaning that even a single RPi might be able to render to a large LED screen in the future. But to do this we first need to understand the protocol they use, which is the focus of this document.

2.1. Previous Research

The protocol has been explored multiple times in the past, but so far without great success.

A simple replay "attack" has been done by Frank E. from mikrocontroller.net^[5], demonstrating the stateless property of the protocol and the initial feasibility of controlling the Linsn RV908 modules through a simple Ethernet port.

In an eevblog forum thread^[6] some users explored the serial protocol to control the controller / sender card, which can be connected to a receiver. Even though this is raw serial, it is relevant in the Ethernet protocol, as can be seen later on. Sadly the research stopped with many questions still unanswered.

The hardware of our receiver module was documented by chubby, although there seem to be multiple revisions of the same module out there. Ours does not have a separate RAM module on board and uses a different FPGA.^[7]

[1] <https://github.com/adafruit/RGB-matrix-Panel>

[2] <https://github.com/hzeller/rpi-rgb-led-matrix>

[3] <https://learn.adafruit.com/fpga-rgb-matrix/overview>

[4] Using a receiver as a dev platform: <https://github.com/q3k/chubby75>

[5] <https://www.mikrocontroller.net/topic/352894>

[6] <https://www.eevblog.com/forum/projects/reverse-engineering-a-chinese-led-screen-control-thing-interesting!/50/>

[7] <https://github.com/q3k/chubby75/issues/20>

Chapter 3. The hardware

The research was done using a modern RV908M32 receiver module by LinsnLED. It has 12 HUB75 outputs, with each being able to control a chain of panels [the exact amount depends on a few factors]. The processing is done by an Analogic FPGA. Overall a single RV908M32 can control up to 1024 x 256 pixels at 12bit HDR and variable FPS. The communication happens over a single 1G Ethernet cable. The board itself is driven over 5V input, but is not responsible for providing power the panels themselves. They seem to have the ability to measure the power usage of the panels, but this has not been verified nor tested.

NOTE

After looking into the protocol we are mostly sure, that the protocol does not change between receiver revisions and as such is fully backwards compatible. Of course hardware limits of the respective receivers apply.

3.1. SPI

An interesting feature of the RV908M32 and RV901^[1] is the existence of an SPI attached flash on the board. Luckily Linsn has not removed nor destroyed the SPI programming header which allowed us to read the FPGA bitstream. The SPI flash can be accessed the following way [from Chubby75^[2]].

The SPI flash holding the configuration memory [U2] is accessible via JP5. The flash I/Os are unbuffered 3.3V, connected through 33Ω resistors. The PROG_B signal has to be pulled LOW to GND [without a resistor] in order to access the SPI flash.

— Chubby75 RV901 hardware description

Table 1. SPI Programming header pinout - adapted from Chubby75^[3]

Shared	SPI Pin	Header Pin	Header Pin	SPI Pin	Shared
CLK	CLK	1	2		
MISO	DO	3	4		5V
		5	6		PROG_B
CS	CS	7	8		
MOSI	DI	9	10	GND	GND

Interestingly all non-volatile configuration is also stored here, meaning that in a large scale system it is plausible to recover and restore the configuration by using the header. This might be useful in cases, where the board does not respond anymore due to misconfiguration, so it is recommended to backup up the data stored there before exploring the hardware.

3.2. Ethernet

Our board uses a Chinese produced 1000BASE-T Ethernet chip called Maxio MAE0621B^[4]. Contrary to statements made by Linsn, the chip supports auto MDI-X, meaning that no Ethernet crossover cable is required. It is otherwise a fairly normal 1000BASE-T Ethernet chip, that supports most common features one would expect.

3.3. FPGA

As mentioned before we were able to read the FPGA bitstream from the flash. Without going into much detail here, we found that our revision of the RV908M32 uses an Anlogic EG4S20BG256. This is in contrast to the Xilinx based FPGA they used as late as 2020. We figured this out, by using the bitstream format description for Anlogic FPGAs by the mmicko/prjtang project^[5]. With this change Linsn was able to remove the external DRAM from the board, as the Anlogic FPGAs have one already integrated, presumably reducing the manufacturing cost.

It is known through the Chubby75 project^[6], that it can be accessed using exposed JTAG contacts. There has been no further research into the FPGA itself as this is not the focus of this project.

3.4. The LED panel

The LED panels we used here are standard HUB75 RGB LED panels. There have been numerous description on their inner workings before. See for example C. Klüser's extensive post on Lushay Labs using a Tang Nano FPGA^[7] to control these panels. For this project we used a few very different panels lying around to explore the receiver and had great success with all of them.

[1] <https://github.com/q3k/chubby75/blob/master/rv901t/README.md>

[2] <https://github.com/q3k/chubby75/blob/master/rv901t/doc/hardware.md>

[3] <https://github.com/q3k/chubby75/blob/master/rv901t/doc/hardware.md>

[4] <http://www.maxio-tech.com/product/12928/12929/12930/13005.html>

[5] https://github.com/mmicko/prjtang/blob/master/docs/architecture/bitstream_format.rst

[6] <https://github.com/q3k/chubby75/issues/20>

[7] WayBack because they broke their images: <https://web.archive.org/web/20240422085913/https://learn.lushaylabs.com/led-panel-hub75/>

Chapter 4. The protocol

WARNING

This document is according to our current understanding of the protocol and is WIP

The protocol is mostly stateless, with a stateful sub-protocol for configuration nested within. It is an OSI 3rd layer protocol, using Ethernet II [DIX 2.0] frames as a carrier medium. Every packet can be split into a header and a data section, with the latter only carrying segmented image frames. Everything else [configuration, discovery, ...] is handled through the header with other protocols muxed into it.

4.1. Ethernet (Physical)

The RV908M32 uses a 1000BASE-T physical medium to transfer the data. As the sender nor the receiver cards mention specific compatibility patterns on their respective store pages it will be further assumed that this holds true for every LinsnLED receiver card. The current understanding is, that faster or slower Ethernet standards are not supported at least on the RV908M32.

Our revision of the RV908M32 supports automatic MDI-X, so no crossover cable is required for a successful communication. There is a note on the LinsnLED webpage telling users to employ a crossover cable, so there might be boards that require it. If it doesn't work, it is recommended to give it a try.

The RV908M32 is otherwise compatible with normal Ethernet and can be plugged into a normal computer supporting 1000BASE-T.

4.2. Ethernet II frame

The data is carried in the payload section of Ethernet II [DIX 2.0] frames using two custom **EtherTypes**, one for each sending direction. As such each EtherType has a specific **payload** size associated with it. See [Table 2](#) for more details.

Table 2. EtherTypes employed by LinsnLED

EtherType	Payload Size	Description
0xaa55	1472 bytes	Sender → Receiver
0xaa56	1482 bytes	Receiver → Sender

NOTE

EtherType is described as big endian.

We will be using here Ethernet II [DIX 2.0] frames in the format described by [Figure 1](#). Preamble and CRC can generally be ignored, as they are automatically generated and stripped by the network hardware. If these two would differ from the spec, the packets would not be received by the corresponding hardware, making the protocol incompatible with standard network gear. Due to this they will not be mentioned going forward.

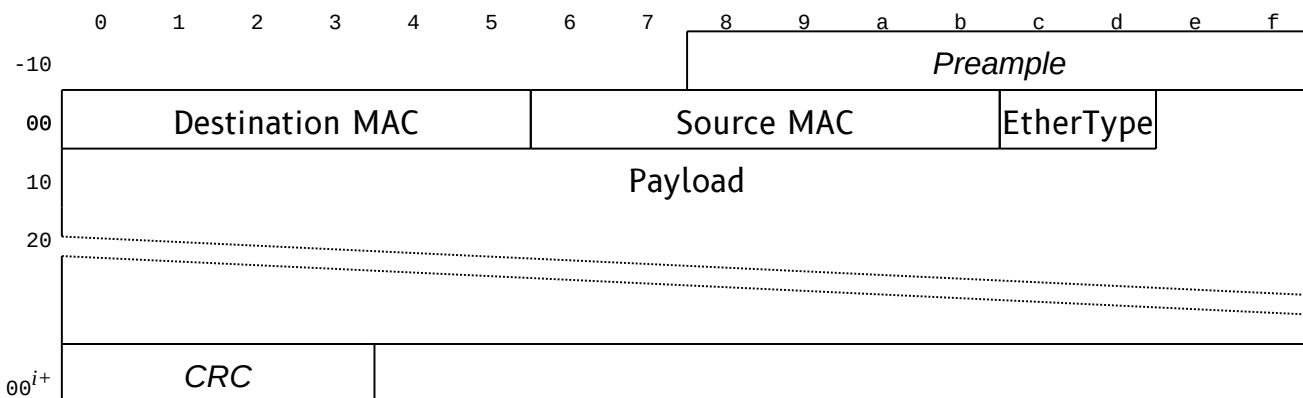


Figure 1. Ethernet II [DIX 2.0] frame. Fields in italics are usually opaque to the user

4.2.1. MAC addressing

The **sender** does not seem to have any limitations imposed on its MAC address. In our research the default MAC of the sender hardware was used which worked without restrictions.

For the **receiver** this does not seem to be true. But TBD

CAUTION

TODO It is currently unknown which MAC the corresponding receivers use for transfer. A single receiver uses `00:00:00:00:00:00`, but it is unknown how this changes with more modules attached sequentially. There was also a random MAC address by the receiver identified once in an old recording from somebody else.

Generally the MAC address discovery happens through a specialized request / response dynamic at the start of the transfer, further described in [Section 4.6](#).

4.3. LinsnLED Ethernet payload

This is where much of the real magic happens. Generally the Ethernet payload of a single Ethernet packet can be split into a **32 byte header** and remainder. For packets of EtherType `0xaa55` the remainder is **1440 bytes of frame buffer data** [see [Section 4.5](#)]. Whereas the current assumption is, that packets of EtherType `0xaa56` do not make use of the 1450 byte long remainder segment [TBD].

CAUTION

TODO For EtherType `0xaa56` only packets of type Mac discovery [[Section 4.6](#)] have been discovered. There might be other messages from the receivers for which this does not hold.

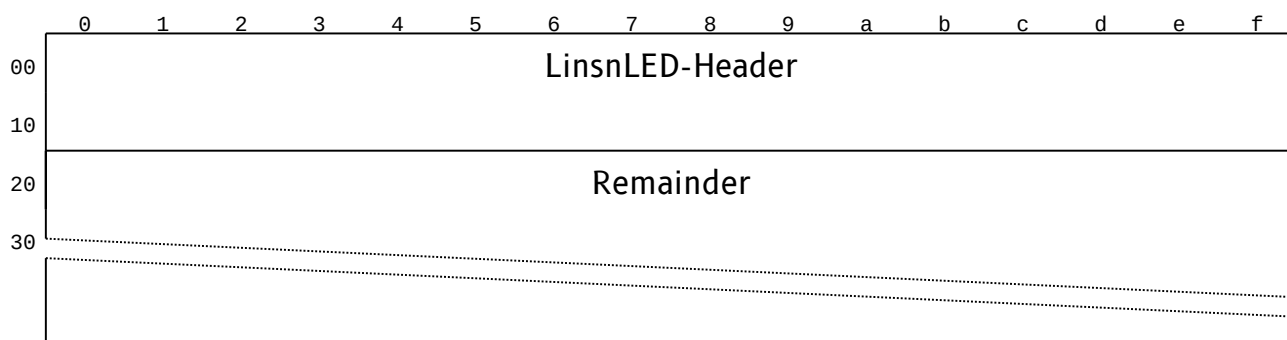


Figure 2. LinsnLED Ethernet payload

4.4. The LinsnLED header

The header format is the same for both EtherTypes. It is always 32 bytes long and can be described according to [Figure 3](#). Numbers are parsed as little endian if not mentioned otherwise.

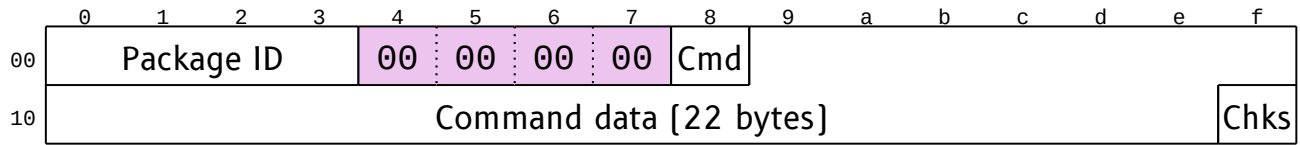


Figure 3. General LinsnLED Header. The fields explicitly mentioned here are always to be found. Semantic of highlighted part is unknown.

CAUTION

TODO As can be seen in [Figure 3](#) the semantics for bytes 4 to 7 are currently not known. So far only zeros bytes have been observed, but there could be a deeper meaning.

4.4.1. Fields

Package ID

This is a 4 byte LE unsigned number. For EtherType `0xaa55` it is directly tied with the frame buffer data of the same package [see [Section 4.5](#)]. A frame is split into chunks and the package ID starts at zero for the first such chunk and increments by one for each following chunk. For EtherType `0xaa56` [from receiver] the field has a different meaning during MAC discovery [see [Section 4.6](#)].

A package ID of zero **enforces** a MAC discovery command [see [Section 4.6](#)].

Checksum (Chks)

The whole header except Package ID and this checksum is verified for correctness using a **single byte** number. Assuming `header[n]` is the `n`-th byte of the 32 byte header and using X_8 to mark the number as an eight bit register using two's-complement for conversion, the checksum is calculated with:

$$\sigma_{checksum} = \left(- \sum_{i=4}^{31} \text{header}[i] \right)_8$$

Command (Cmd)

Command is a **single byte** field. So far three possible values are known. The corresponding value changes the way the command data field must be interpreted. This is detailed in the corresponding sections. See [Table 3](#) for a list, description and reference of each.

Table 3. All currently known command field values

Value	Description	Reference
<code>0x96</code>	Used to announce MAC addresses	See Section 4.6
<code>0x61</code>	Used to transfer configuration data to a receiver	See Section 4.7
<code>0x00</code>	Indicates that the header is mostly zeros. [most common one]	See below

The cmd value `0x00` is special as it is the nothing command. The command data field [and panel index] will be filled with zeros presumably because they will not carry any information. It is by far the most common one, as it is the default command used during frame transfers.

Command Data

See Cmd field [Section 4.4.1.3].

4.5. Frame buffer data

This is **exclusively** used by the sender [EtherType **0xaa55**] and is always **1440 bytes** long. So far it seems to be the only remainder type supported. [TBD]

As a single frame does not fit into the 1440 byte payload, instead it is split up and transferred in chunks of the same size. The data is stitched together according to the **package ID attribute** in the header, whereby a 1440 byte long chunk starts at offset **1440 byte * segment-number** in the final frame.

The transferred image generally has the form of 512 * 256 px [width x height] and can [for interpretation] be split into 256 * 256 px sub-images [left and right]. Left always seems to contain an image, whereas the right side has been observed with and without an image. No image here means that it was filled with zeros. Interestingly the first image row was always filled with zeros so far, the purpose of that is unknown. The only observed encoding was uncompressed 24 bit BRG [blue/red/green] data.

CAUTION

TODO There are a lot of unknowns about this part currently, especially how it is configured / announced to the receiver what the exact data layout is. As mentioned above 24 bit BRG has been observed, but it is known that our receiver is theoretically capable of 12 bit HDR and that other channel orders might be possible. Currently, there is no information available how this would present itself in the data.

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
00	Package ID				00	00	00	00	00	00	00	00	00	00	00	00
10	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
20	BB	RR	GG	BB	RR	GG	BB	RR	GG	BB	RR	GG	BB	RR	GG	...

Figure 4. Schematic example data chunk for frame buffer including LinsnLED header using zero command

4.6. MAC discovery

This requires the command field in the header to be **0x96**.

During a usual protocol cycle there are two places where this command type is being used. LinsnLED sender applications use it **during the "initialization" phase** to discover the MAC addresses of all attached receiver modules. Later on this command is employed **to mark the start of a new frame**. The receiver always answers in ideal circumstances [described in Section 4.6.2].

NOTE

The protocol is stateless, so there is no **initialization phase** in the strict sense. Still the controllers need to discover all modules in order to address them, which is meant here as "initialization" phase.

The biggest difference between these two use-cases is the receiver MAC address. For the general discovery of receivers the Ethernet destination is broadcast, whereas later on [to mark a new frame] it is addressed at a specific module. For broadcasted packages the content of the

remainder / frame buffer data seems to be irrelevant.

4.6.1. Data layout

The exact interpretation of the data segment is currently unknown. There is a long non-zero segment whose value has not changed in observed transfers, so the exact meaning is unknown. Current naive assumption is that it is just a static piece of data. At the end one can find the 6 byte long MAC address of the sender.

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
00	Package ID				00	00	00	00	96	00	00	00	85	1f	ff	ff
10	ff	ff	00	00	00	00	00	00	00	Sender MAC						Chks

Figure 5. MAC discovery packet header. Exact meaning for the highlighted part is unknown. Assumed to be static [TBD]

CAUTION

TODO It is very unlike the current understanding of the protocol that there are [relatively long] segments of data which are void of any meaning. So be on the lookout for anything weird going on here, it could help the RE process.

4.6.2. Receiver packet

The answer by the receiver is very different. It still adheres to the general header data layout as described in [Section 4.4](#), but the **meaning of the packet ID field changes** a bit. The command byte is zero.

As the observed value so far is relatively high, it could describe the maximum supported package ID, the maximum buffer storage capacity, TBD

The remainder segment seems to be irrelevant in this case.

CAUTION

TODO Unknown what the packet ID field does in this case.

4.7. Configuring the system

This requires the command field in the header to be **0x61**. In stark contrast to the rest of the protocol, the [sub-]protocol described here is **stateful**.

This sub-protocol presumably stands out so much from the rest, because it is also spoken with the controller card [the element normally between the receiver and user agent] in a slightly adapted form. In these cases it is a serial protocol, which could explain a few of the design choices they have taken here.

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
00	Package ID				00	00	00	00	61	Panel IDX	Info	Address			...	
10	Data															Chks

Figure 6. System configuration packet header. The highlighted part is further detailed in [Figure 7](#)

7	6	5	4	3	2	1	0
Type				is_data?	Config Pkg ID		

Figure 7. The structure of the info byte [in bits]

4.7.1. Fields

Panel Index

This is a 2 byte LE unsigned number. It declares the receiver that will process this configuration change packet. If this is a general configuration change meant for all receivers, this value will be 0.

CAUTION

TODO How the addressing actually works, needs to be explored further. One big point is that the receivers don't have unique static IDs in their own right, so how does this work?

Type

After trying to understand type as a bit-field for some time and having trouble making any sense out of the emerging patterns, we shifted to interpreting it as a 4 bit integer enum with four defined values. These describe mostly how the payload [data field] will be interpreted, and are a big part of the statefulness of the protocol. The values are described in [Table 4](#).

Table 4. The enum values for the config command type field

Value	Description
1	temporary data
5	state configuration
8	permanent data
9	permanent operation

We distinguish between permanent and temporary data, as the configuration can also be simply written to RAM instead of SPI. In the UI this allows the user to experiment with different configurations without overwriting a known good setting.

The permanent operation are actions that can be executed on the non-volatile storage. Only block erase is known so far.

is_data?

This field is a simple bit flag, declaring that the payload is actual data and not meta-data which would describe how the following packets should be interpreted and used.

This value is true if and only if `type=1` or `type=9`.

Config Pkg ID

This field is independent of the carrier packet ID.

A single [sub-]packet is repeated multiple times [independent of the carrier packet] before the next one is being sent. This might be a mitigation against possible packet loss, as there are no ACK mechanisms in the protocol.

To filter these repeated packets, the protocol uses the 3 bit config packet ID. For each new unique packet this value is increased by one and looping back to 0 after reaching 7.

NOTE | This value is stable between configuration sequences.

Address

This is a 3 byte long big-endian integer. The integer specifies the data segment in [virtual] memory the payload should be applied to. In cases where the type enum assumes a configuration value [`type=5` or `type=9`], the address could also be a static value or an address with specific limitations (e.g. must be a multiple of 4096).

Data

This 16 byte field or raw data contains can be one of the following:

- configuration data
- further describes the command to be executed
- static values (assumed to be gibberish and non-important)

4.7.2. State machine

Figure 8 shows the state transition diagram for the configuration protocol. Next to every transition the conditions for it are declared as fields (and their values) that the config packet needs to match. To keep the diagram readable only unique packets are considered here (packets with differing config pkg id).

The states in the sub-diagrams are meant to be repeated until the end packet is transferred, starting from the start state with each new packet. With this a large chunk of memory can be written in a single transfer.

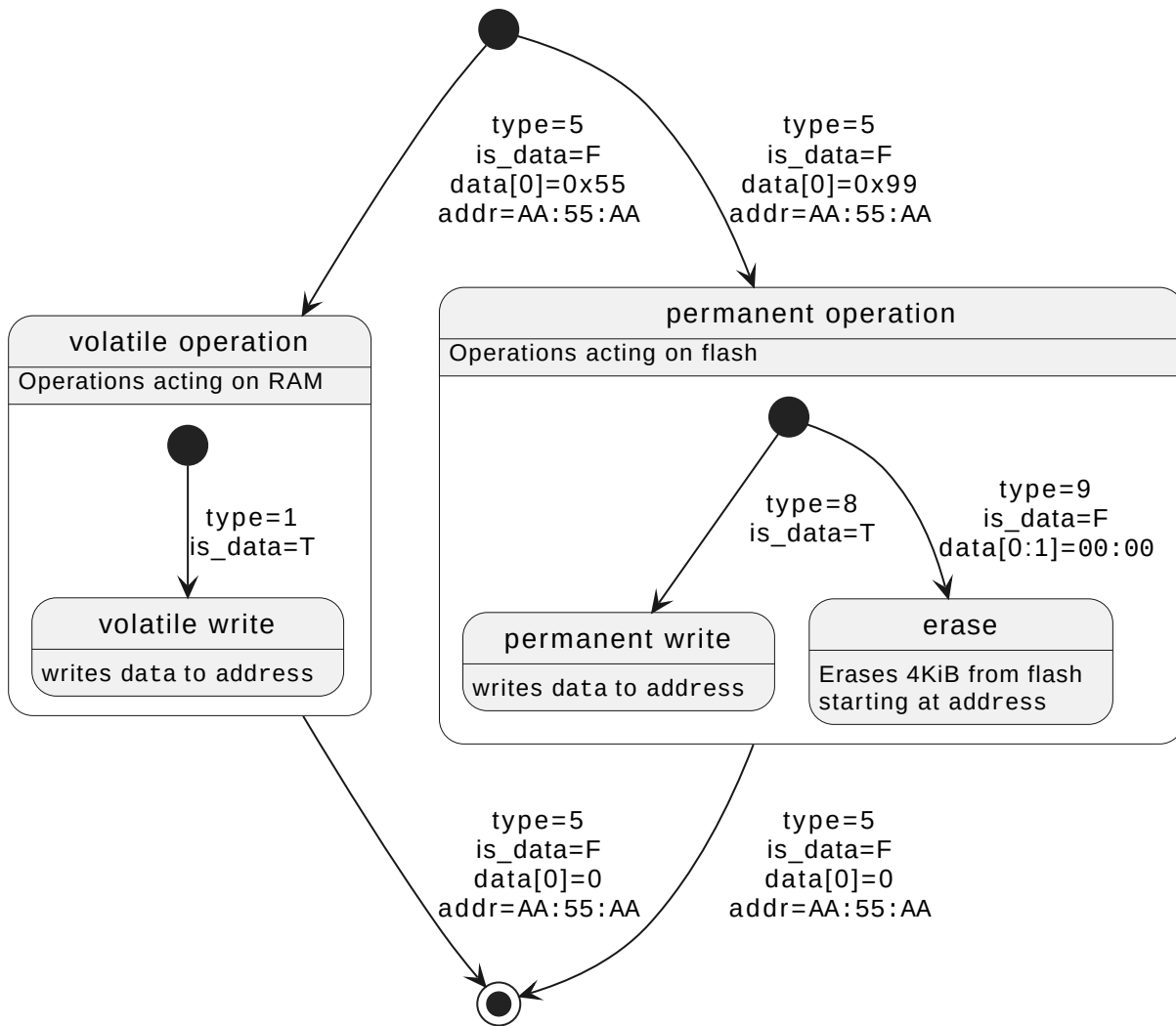


Figure 8. State transition diagram for the configuration message protocol

4.7.3. Possible Payload values

In [Section 4.7.1.6](#) multiple payload types were mentioned, without going into much detail which values they could take exactly. [Section 4.7.2](#) partially mentions possible data field values, but for the sake of brevity only the most relevant parts are shown. This section should clear that up a bit.

The possible values the data field assumes can be categorized using the type field.

Temporary/Permanent Data (type=1 or type=8)

The data field here is exactly what will be written into memory. It's 16 bytes of data that will be written into flash or RAM starting from the location specified in the address field.

State Configuration (type=5)

Here the first byte of the data field is relevant, the remaining 15 bytes are filled with the value 0xd8. Depending on whether this is a volatile, permanent start of transition or end of transition, the first byte is 0x55, 0x99 or 0x00 accordingly.

Permanent Operation (type=9)

In this case the address field shall be a multiple of 4 KiB [0x1000 bytes]. The first two bytes of the data field are interesting, the remaining 14 bytes are filled with 0xd8.

Currently, only one command is known, which is the erase function. In this case the first two bytes will assume the value `00:00`.

Chapter 5. LinsnLED Memory layout

Configuration of receiver cards happens by writing to their [volatile / non-volatile] memory directly. For the LinsnLED protocol the semantic of a specific address doesn't change [no position independent configuration], which eased our RE effort. This chapter defines the memory map. On how to change these values and write them to the receiver cards see [Section 4.7](#).

CAUTION

TODO Note that not a lot of fields are known so far, there's a lot going on in the memory, but only a [small] subset has been understood so far.

5.1. Fields

5.1.1. Offset **0x000501**: Cascade Direction (CasD)

length: 1 byte[s]

CAUTION

TODO It does something, but it is hard to put into words what

Table 5. Possible values for cascade direction

Value	Meaning
0x20	From right to left
0x60	From left to right
0xA0	From bottom to top
0xE0	From top to bottom

As you can see from [Table 5](#), this value might also just be a 3bit value. Unsure if this field is actually 8bit long.

5.1.2. Offset **0x000508**: Duty Cycle (DuCy)

length: 1 byte[s]

Although the exact meaning is unclear, it is known how the value is derived. In it's simplified form it's the duty ratio times the scan clock frequency.

Assuming D_R the duty ratio [in percent], s the value of the scan clock lookup, then the duty cycle [$D_C \in [1, S) \subset \mathbb{N}$] can be expressed as:

$$D_C = D_R \cdot (S + 1) / 100$$

5.1.3. Offset **0x000509**: Scan Clock Lookup (ScCl)

length: 1 byte[s]

In the original LedSet application, this value is derived from a lookup table using the selected scan clock in MHz as an index. The mapping can be found in [Table 6](#). Except for the outlier in 0.6MHz, this table is an inverse exponential curve.

WARNING

TODO This is only applicable for some chips, others use a different lookup table

Table 6. Scan Clock Lookup Table from MHz to real value

Value	Raw Value	Value	Raw Value	Value	Raw Value	Value	Raw Value
30.0 MHz	0x04	12.5 MHz	0x0b	6.5 MHz	0x16	3.0 MHz	0x31
25.0 MHz	0x05	11.5 MHz	0x0c	6.0 MHz	0x18	2.5 MHz	0x3b
21.4 MHz	0x06	10.7 MHz	0x0d	5.6 MHz	0x1a	2.0 MHz	0x4a
18.8 MHz	0x07	10.0 MHz	0x0e	5.0 MHz	0x1d	1.5 MHz	0x77
16.7 MHz	0x08	9.4 MHz	0x0f	4.5 MHz	0x20	1.0 MHz	0xef
15.0 MHz	0x09	8.3 MHz	0x11	4.0 MHz	0x25	0.6 MHz	0x02
13.6 MHz	0x0a	7.5 MHz	0x13	3.5 MHz	0x2a		

5.1.4. Offset 0x000512: Drive IC (DIC)

length: 1 byte[s]

This declares the chipset the attached led matrix employs. It is a simple lookup-table with 180 entries [see Table 7].

Table 7. The drive ic and their mapped values

Name	Value	Name	Value	Name	Value	Name	Value	Name	Value
General	0	LXY28161/165	1	LXY28163	2	LXY28380	3	LXY28169	4
MBI5034	5	MBI5031	6	MBI5030	7	MBI5036	8	MBI5039	9
MBI5042	10	MBI5050	11	DM13H	12	TC62D722	13	MBI5224	14
MBI5051	15	MBI5052	16	MBI5053	17	MY9266	18	MY9268	19
MY9269	20	MBI5045	21	MBI5041B	22	MBI5151	23	MBI5152	24
MBI5153	25	SUM2030	26	SUM2032	27	SUM2017	28	MY9221	29
MY9231	30	SUM2019	31	MY9366	32	MBI5124	33	MBI5124DP WM	34
MY9868	35	MY9862	36	MBI5166	37	SM16136	38	SM16158	39
SUM2028	40	MBI5043	43	MBI5155	44	ICN2027	45	SM16159	46
DM6902	47	SUM2017T	48	SUM2033	49	SM16017S	51	MBI5353	52
LXY345General	56	LXY345+LXY163	57	LXY345+LXY166	58	LXY345+LXY380	59	ICN2038S/2045	61
ICN2038S_T/2045_T	62	ICN2053/2050/2153	63	MBI5252	64	SUM2130	65	ICN2038	66
SM16207S	67	ICN2037	68	WS2812	69	GW6205	70	UCS8903	71
UCS9812	72	UCS1903b	73	XT1511	74	MBI5051B	75	SM16237S	77
ICN2058	78	ICND2055	79	FM6182	80	MY9373	81	SM16259	82

Name	Value	Name	Value	Name	Value	Name	Value	Name	Value
ICN2028	83	SM16227S	84	FM6153	85	FM6124	86	SM16237D S	87
ICND2065	88	MY9866	89	FM6126C	90	SUM2035	91	SUM2117	92
SUM2017T D	93	ICND2110	94	SM16188B	95	ICND2046/I CND2049	96	MY9866_T	97
SM16359	98	RT6848	99	ICND2112	100	ICND2200	101	CF6512	102
ICND2163	103	SUM6082	104	MBI5253	105	MBI5359	106	LS9917	107
LS9920	108	SK6812	109	SK9816	110	MBI5325D	111	MBI5325D_ DPWM	112
MBI5325S	113	MBI5325S_ DPWM	114	MBI5850	115	MBI5759	116	ICND2047	117
SM16169S	118	FM6127	119	MY9553	120	SM16218	121	HBS1923	122
FM6353	123	FM6356	124	HBS1910	125	FM6555	126	MBI6023	127
RT5965	128	YY6018	129	UCS5603	130	FM6565	131	FM6363	132
ICND2076	133	MY9235	134	MBI5251	135	SM16369S	136	DP5525	137
AXS6018	138	HX8055	139	MY9748	140	CS2033	141	ICND2159	142
RT5966	143	SK9810	144	WS2816B	145	FM6047	146	RT5967	147
CNS7153	148	TX1816Z	149	HX8864	151	CS2017	152	DP3264	153
MBI5264	154	ICND2210	155	DP3246	156	MTS01	157	D2618	158
SM16380	159	SM16389	160	TX1816ZC	161	RT7920	162	FM6129	163
MBI5268	164	MBI5254	165	SM16388	166	SUM6086	167	RT5938	168
MBI5850B	169	TM1905	170	CFD455A	171	ICND2069	172	CFD135A	173
DP3265	174	ICND2263	175	ICND1063	176	LS9918	200	LS9929	204
LS9930	205	LS9919	206	LS9935	207	LS9926	208	LS9935B	209
LS9928	210	LS9936	211	LS9931	212	LS9929C	213	LS9933	222

5.1.5. Offset **0x000518**: ? & Image Control (ImCn)

length: 1 byte[s]

This is a 4 bit value. The lower part is the so called image control, the upper is unknown.

Image control describes what should be displayed, if no new frame data arrived. See [Table 8](#) for what each value means.

Table 8. Image Control values

Value	LedSet Name	Description
0	Black	Display is turned off
1	Random	Freezes the last frame
2	Image	Displays a preset image

Value	LedSet Name	Description
3	Auto Test	Executes an automatic test sequence which can be configured.

WARNING

TODO It is currently unknown, how Random and Auto Test are configured. Especially the image modus wasn't successfully tested so far.

5.1.6. Offset **0x000560**: Unknown 1

length: 32 byte[s]

It is currently unknown what it means. It seems to be a 16bit integer array [16 values], which depends on our row blanking time.

5.1.7. Offset **0x0006a0**: Data Exchange (part I)

length: 32 byte[s]

This configures the order of each image data segment in relation to the final output. It is a mapping with 8 bit values, where the key is the position in the array.

A data segment here is a display subsection, that is covered by a single scan line. So for a 64px high 32S display, there would be two data segments.

This configures only the first 32 data segments, the array is continued in [Section 5.1.16](#) for the remaining 96 possible configurable segments.

5.1.8. Offset **0x000709**: Afterglow time (lower byte) (Aft-)

length: 1 byte[s]

The afterglow time. Unit is unknown. This value is split into a lower and upper byte, which are at different positions. See [Section 5.1.11](#), for the upper byte.

5.1.9. Offset **0x000760**: Newline Time (Newline)

length: 2 byte[s] / byte order: little endian

The unit is unknown. Limit seems to be afterglow - 1.

5.1.10. Offset **0x000762**: Max afterglow time (MaxAft)

length: 2 byte[s] / byte order: little endian

This value represents the maximum possible afterglow time. It $[M_A \in \mathbb{N}]$ is derived from the [actual] row blank time $[T_R]$ using the formula:

$$M_A = T_R \cdot 100 / 667$$

5.1.11. Offset **0x000764**: Afterglow time (upper byte) (Aft+)

length: 1 byte[s]

Upper byte of afterglow time. See [Section 5.1.8](#) for more information.

5.1.12. Offset 0x001002: Start X

length: 2 byte[s] / byte order: little endian

Starting X coordinate of the subimage the receiver displays. The controller always sends a large continuous frame, from which each receiver extracts their own subimage.

5.1.13. Offset 0x001004: Start Y

length: 2 byte[s] / byte order: little endian

Starting Y coordinate of the subimage the receiver displays. The controller always sends a large continuous frame, from which each receiver extracts their own subimage.

5.1.14. Offset 0x001006: Width

length: 2 byte[s] / byte order: little endian

The display width, as configured in the UI.

5.1.15. Offset 0x001008: Height

length: 2 byte[s] / byte order: little endian

The display height, as configured in the UI.

5.1.16. Offset 0x0011a0: Data Exchange (part II)

length: 96 byte[s]

See [Section 5.1.7](#) for more details.

5.1.17. Offset 0x002000: Gamma Curve (1st channel)

length: 512 byte[s]

Gamma curve for the first color channel. This is 256 element long list of LE 16 bit values that describe the gamma curve mapping. The index of an element maps to the specific color value [starting at 0].

5.1.18. Offset 0x002208: Global Brightness (GoBr)

length: 4 byte[s]

This is a four element list, with each element describing the brightness [from 0 to 255] for a single channel. 255 is full brightness.

We are unsure what color the fourth channel is, especially as it isn't exposed by the UI, but it might be a way to future proof the system or for some esoteric four color LED matrix Linsn once supported.

5.1.19. Offset **0x00227f**: Card Mode (CaMo)

length: 1 byte[s]

This is relevant for the data exchange [see [Section 5.1.7](#)], as it essentially configures how many data segments exist [aka the length of the array]. It is a simple lookup value [see [Table 9](#)]

Table 9. Card mode values and their meaning

Value	Description
0	Normal [16 segments]
1	16 segments for RV907
2	20 segments
3	20 segments for RV907
4	24 segments
5	24 segments for RV905
6	28 segments
7	32 segments
8	16 serial segments for RV907
9	32 serial segments
10	64 serial segments
11	128 serial segments

5.1.20. Offset **0x002800**: Gamma Curve (2nd channel)

length: 512 byte[s]

See [Section 5.1.17](#) for more info.

5.1.21. Offset **0x002a00**: Gamma Curve (3rd channel)

length: 512 byte[s]

See [Section 5.1.17](#) for more info.

5.1.22. Offset **0x002c00**: Gamma Curve (4th channel)

length: 512 byte[s]

See [Section 5.1.17](#) for more info.

5.1.23. Offset **0x003003**: RCG file path

length: 512 byte[s]

This is the file path of the RCG file. It is simply a 8 bit char c-string. I'm really unsure why it

must be a constant in our RCG file but here we are. This value is displayed in the UI, so we can display fun things with it at least.

Exact length is unknown, but it is not more than 512 bytes.

5.1.24. Offset **0x003207**: Max width

length: 2 byte[s] / byte order: little endian

The maximum width that the receiver supports using the current configuration. This could be derived from other values in the RCG, but as that would be quite complicated [and currently unknown how one would do that], it's easier to just read this value instead.

Is zero, if width is a free parameter without restrictions.

5.1.25. Offset **0x003209**: Max height (MaxHeight)

length: 2 byte[s] / byte order: little endian

The maximum height the receiver supports using the current configuration. If zero, the height is a free parameter. See [Section 5.1.24](#) for more details.

5.1.26. Offset **0x00320b**: Duty ratio (DuRa)

length: 2 byte[s] / byte order: little endian

The duty ratio in percent.

5.1.27. Offset **0x00320d**: Row blanking time (RoBT)

length: 2 byte[s] / byte order: little endian

The actual row blanking time as shown in the UI. This value is only set, when the value behind it is larger than 50 by LedSet [unsure why].

5.1.28. Offset **0x00341f**: Checksum (Chk)

length: 4 byte[s] / byte order: little endian

The 4 byte file checksum for everything that happened before this point. This value is only generated when the RCG is saved to disk and is verified during loading.

It can be calculated using the following python function:

```
def checksum(file: bytes) -> int:
    return sum(file[0:0x341F]) # in python this excludes 0x341F
```


5.2. Memory Table

Address	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
*																
000500		CasD							DuCy	ScCl						
000510			DIC						ImCn							
*																
000560	Unknown 1															
000570	...															
*																
0006a0	Data Exchange (part I)															
0006b0	...															
*																
000700										AfT-						
*																
000760	Newline		MaxAfT		AfT+											
*																
001000			Start X		Start Y		Width		Height							
*																
0011a0	Data Exchange (part II)															
* (cont)																
0011f0	...															
*																
002000	Gamma Curve (1st channel)															
* (cont)																
0021f0	...															
002200										GoBr						
*																
002270																CaMo
*																
002800	Gamma Curve (2nd channel)															
* (cont)																
0029f0	...															
002a00	Gamma Curve (3rd channel)															
* (cont)																
002bf0	...															
002c00	Gamma Curve (4th channel)															
* (cont)																

Address	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
002df0	...															
*																
003000				RCG file path												
* (cont)																
003200	...							Max width		MaxHeight		DuRa		RoBT		
*																
003410																Chk
003420	...															